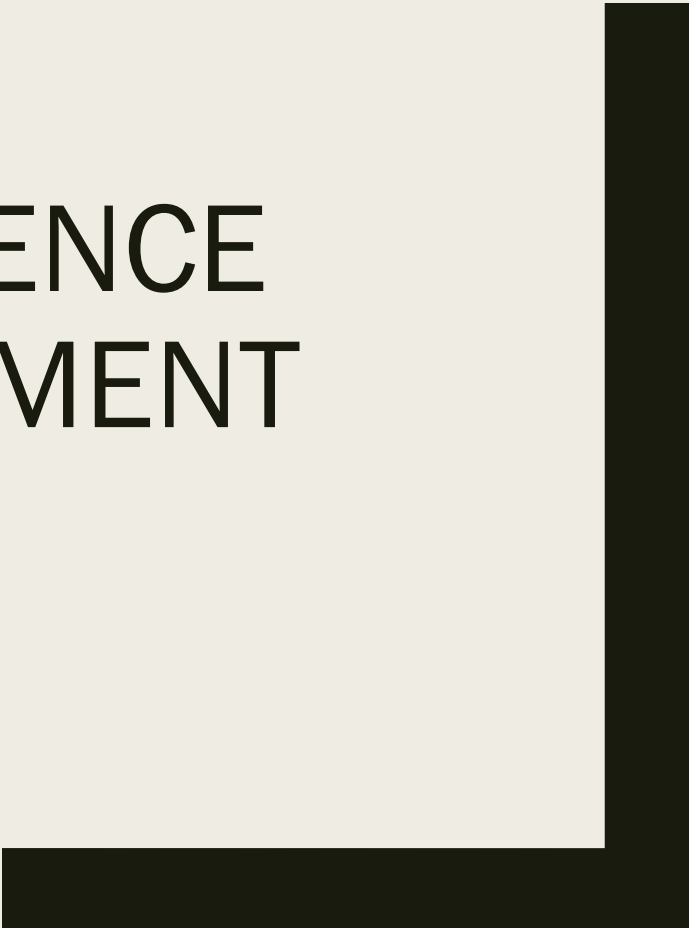




ARTIFICIAL INTELLIGENCE SOFTWARE DEVELOPMENT

CST8510 Week 6
Dr. Hari M Koduvely



Agenda for Today

□ Theory: 5:30PM – 7:30PM

- Model Deployment
- Batch Vs Online
- Model Compression

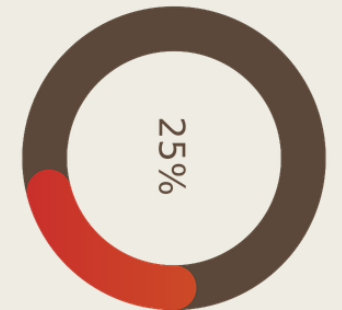
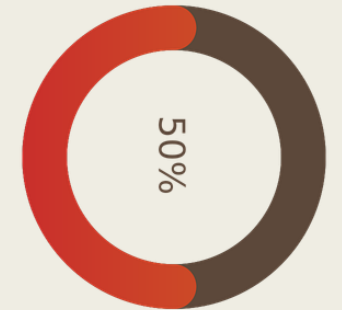
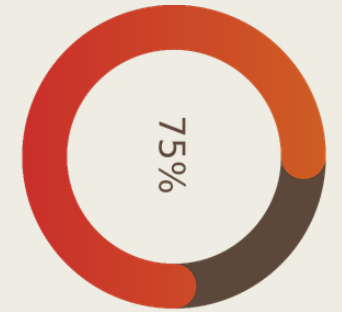
□ Lab: 7:30PM – 9:30PM

- Standup Meetings



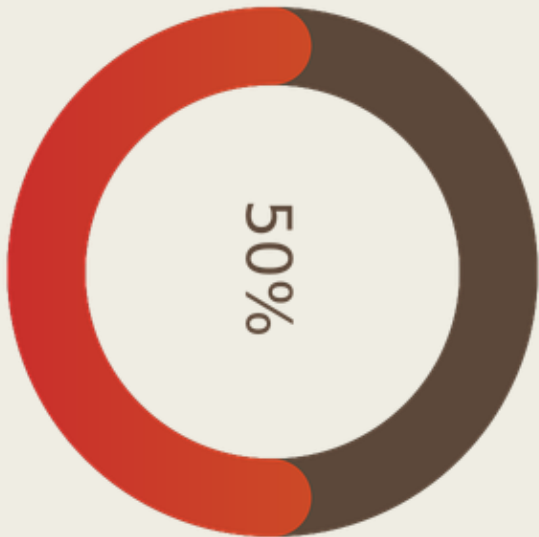
Model Deployment

- ❑ Deploying ML Model can be very different from that of traditional software.
- ❑ This can create some myths among people with no experience.
- ❑ Let's look at some of these myths.



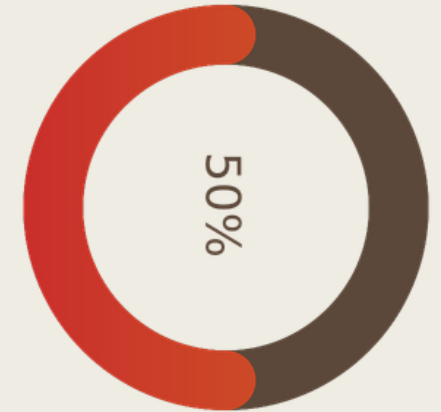
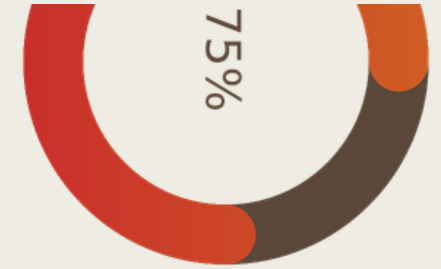
Model Deployment Myths

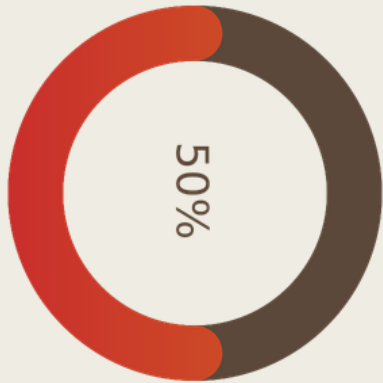
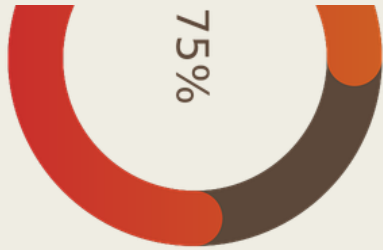
- ❑ Only a very few ML models are deployed at a time.
- ❑ Case study from Uber.
- ❑ Models for:
 - Ride demand
 - Driver availability
 - Estimated time of arrival
 - Dynamic pricing
 - Fraudulent transaction
 - Customer churn
 - ...200+ models



Model Deployment Myths

- ❑ Model performance remains the same
 - Data distribution shifts over a time
 - This result in performance degrade over time
 - Model performance needs to be tracked
 - Models needs to be trained periodically

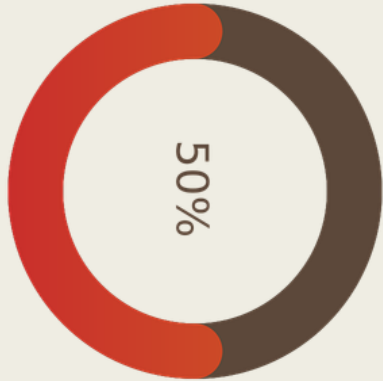
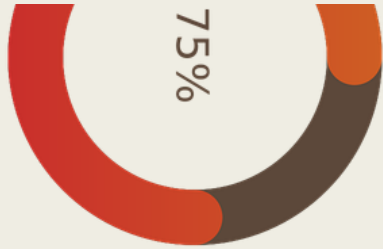




Model Deployment Myths

❑ Models Don't Need Frequent Update

- Retraining frequency depends on the type of problem and domain.
- It can be from every 10 minutes to days and months
- Retraining frequency is high for models:
 - Deployed in real-time systems
 - Used for critical decisions, e.g. fraud detection or medical diagnosis.
 - Used in applications where data is changing rapidly



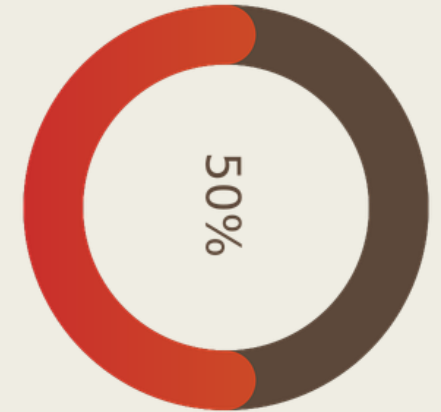
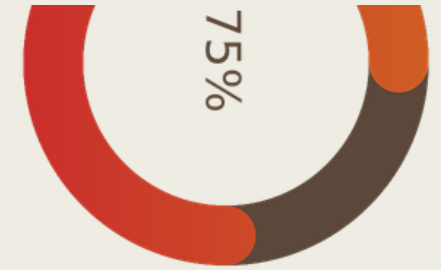
Model Deployment Myths

❑ Models Don't Need Frequent Update

- Retraining frequency is low for models:
 - Used in less critical applications, such as product recommendations or website personalization
 - Used in applications where data is relatively stable, such as in natural language processing or image recognition.

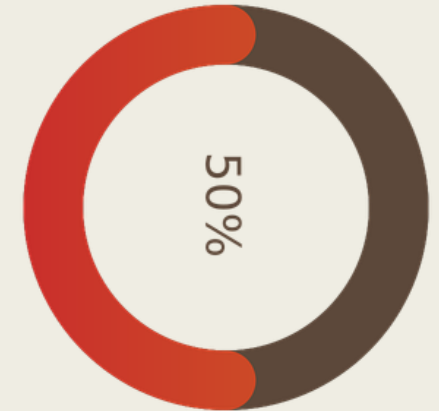
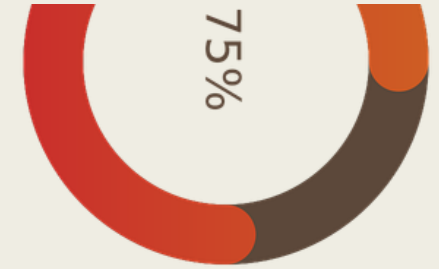
Model Deployment Myths

- ❑ Most ML Engineers Don't Need to Worry About Scale
 - Except those works for Google, Amazon, FB etc.
 - You can have a startup with 100+ employees
 - But having products with millions of customers
 - E.g. Open AI, Slack



Model Deployment Myths

- ❑ Scaling means increasing the number of machines
 - **Vertical Scaling:** Increasing the resources of a single machine (adding more memory or CPU capacity)
 - **Horizontal Scaling:** Adding more machines to a cluster to share the load and handle more requests.
 - **Auto-scaling:** Number of machines is automatically adjusted based on the load. Done using cloud services such as AWS auto-scaling groups or Kubernetes HPA (Horizontal Pod Autoscaler).
 - **Microservices:** System is split into a set of small, independent services, each of which can be scaled independently. Done using service meshes such as Istio or Linkerd.
 - **Hybrid Scaling:** Combination of both vertical and horizontal scaling.



Batch Vs Online Predictions

- ❑ Very important decision to make before deployment
- ❑ 3 types of deployments:
 - *Batch prediction using only batch features.*
 - *Online prediction that uses only batch features.*
 - *Online prediction that uses both batch and streaming features.*



[This Photo](#) by Unknown author is licensed under [CC BY](#).



Online Predictions

- ☐ Predictions are generated and returned as soon as requests arrive.
- ☐ E. g. online translation.
- ☐ Requests are sent to prediction service via RESTful APIs.
- ☐ They are called Synchronous Predictions.

Batch Predictions

- ☐ Predictions are generated periodically or based on some trigger.
- ☐ Predictions are stored in some DB and retrieved when needed.
- ☐ E. g. Netflix Recommendations.
- ☐ Also called Asynchronous Predictions.



Batch Vs Online Predictions

Using only Batch Features

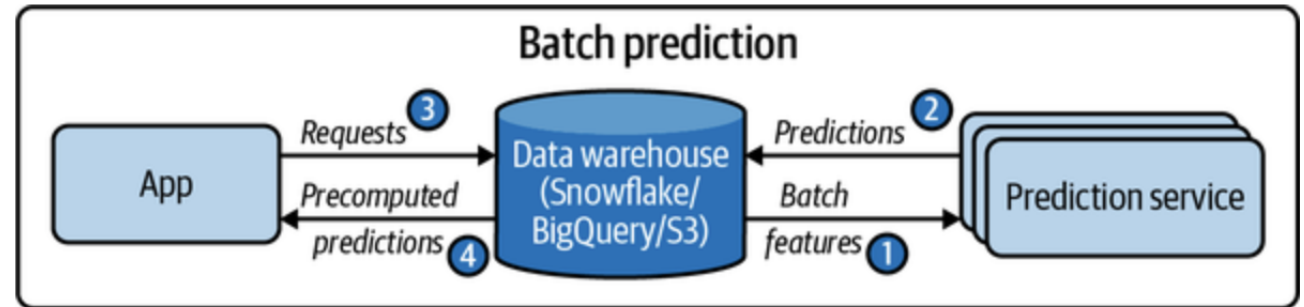
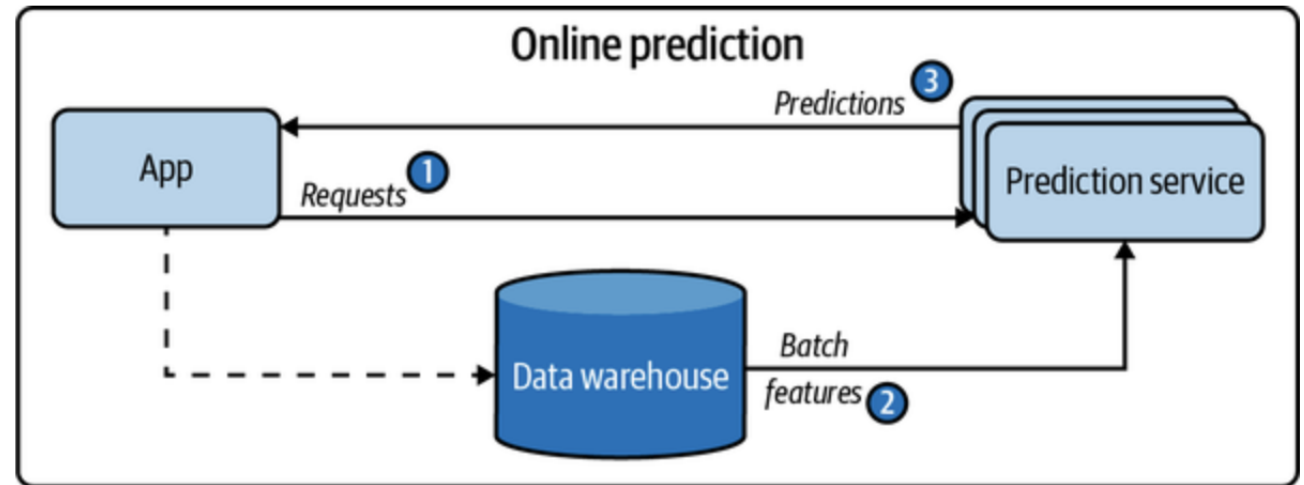


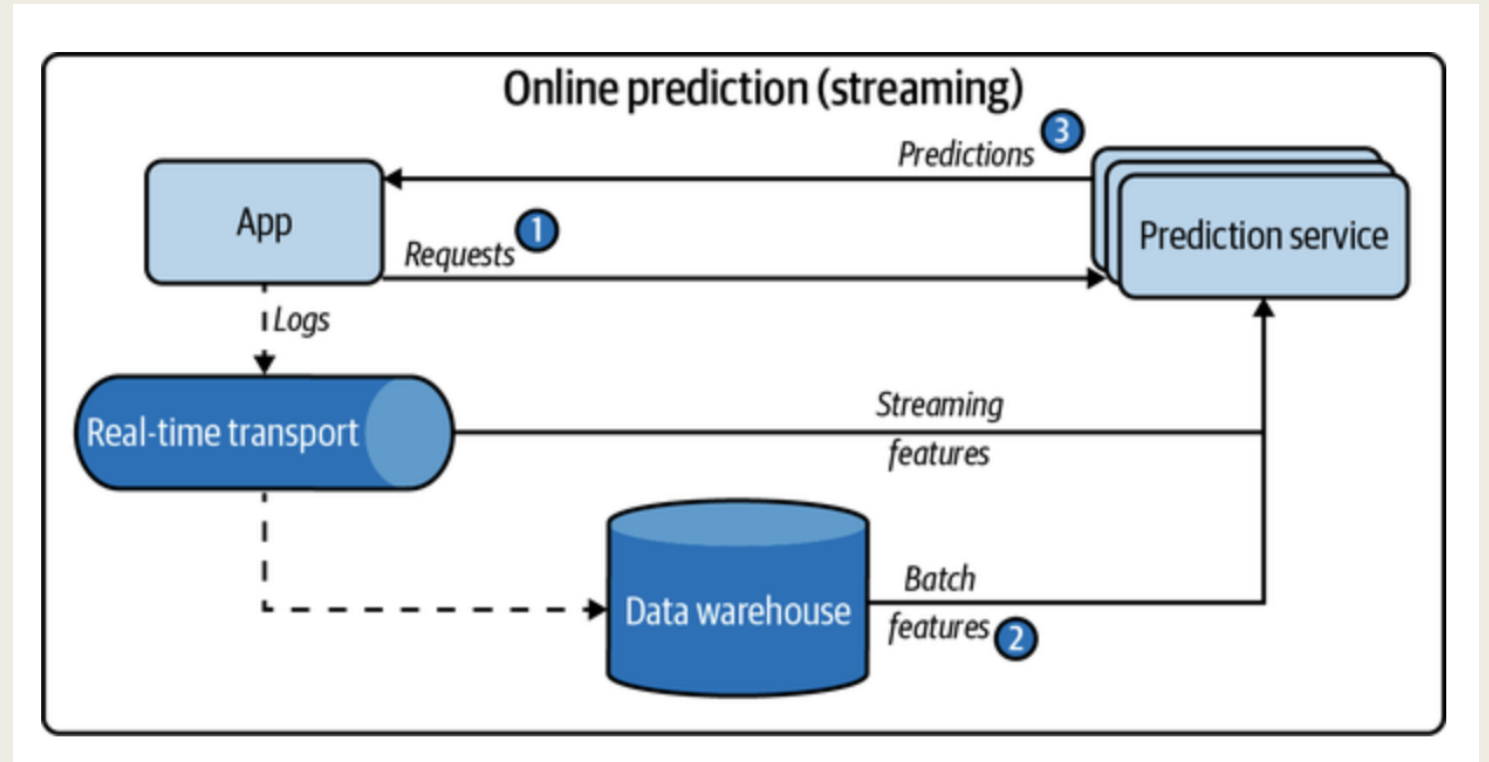
Figure 7-4. A simplified architecture for batch prediction



Source: Designing Machine Learning Systems, OREILLY

Online Predictions

Using both Batch
and Streaming
Features



Source: Designing Machine Learning Systems, OREILLY



Batch Predictions – Merits vs Demerits

- ☐ Batch predictions have low "perceived" latency.
- ☐ They can miss the current context.
- ☐ Batch inference can be made very efficient.
- ☐ Need to know the input in advance
- ☐ Predictions could be a waste.



Online Predictions – Merits vs Demerits

- ☐ Can capture current context.
- ☐ Both streaming and batch features can be used.
- ☐ Input is available with the request.
- ☐ Generate inference only if requires (no waste).
- ☐ But need to build additional infrastructure.
- ☐ Latency could be an issue.

Online Predictions – Reducing Latency

□ Three approaches:

- Do inference faster – Code optimization
- Make the model smaller – Model compression
- Deploy faster hardware



Model Compression

- ❑ Process of making a model smaller in size (bytes).
- ❑ A smaller model would run faster (lower memory requirements).
- ❑ Also, it can be deployed on edge devices.
- ❑ Growing number of open-source projects (168 currently)
<https://awesomeopensource.com/projects/model-compression>



This Photo by Unknown author is licensed under [CC BY-SA-NC](#).

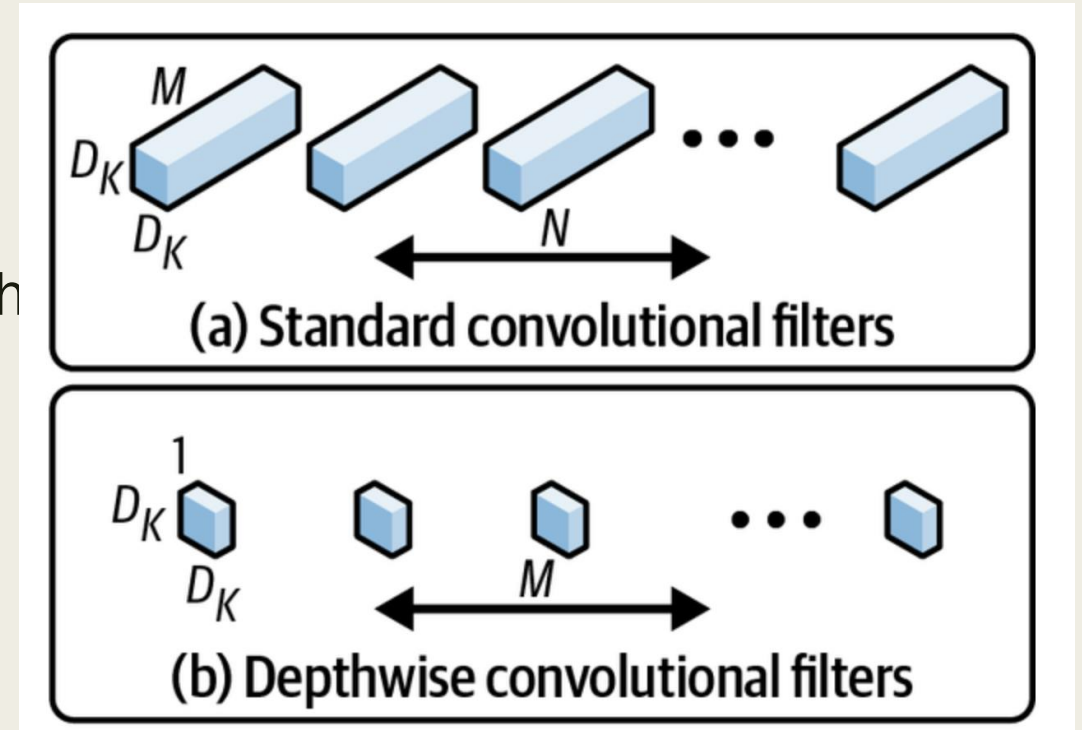
Model Compression



- ❑ Mainly 4 types of techniques for model compression
 - *Low-rank optimization*
 - *Knowledge distillation*
 - *Pruning*
 - *Quantization*

Model Compression - Low Rank Factorization

- ❑ Replace high dimensional tensors with low dimensional tensors.
- ❑ e.g. *Compact Convolutional Filters*
 - *3 X 3 convolution filters are replaced with 1 X 1 filters in SqueezeNet*
 - *Similar accuracy on ImageNet dataset with 50% less parameters compared to AlexNet*



Source: Designing Machine Learning Systems, OREILLY



Model Compression – Knowledge Distillation

- ❑ A smaller model (student) is trained using a larger model (teacher)
- ❑ Smaller model is used for deployment
- ❑ e.g. DistillBERT
- ❑ Reduces the size of BERT by 40%
- ❑ Still maintains 97% NLU capabilities
- ❑ And 60% faster

Model Compression – Knowledge Distillation

- ❑ Why this better than training student model directly?
 - Transfer learning
 - Regularization
 - Learning from intermediate representation
 - Improved generalization

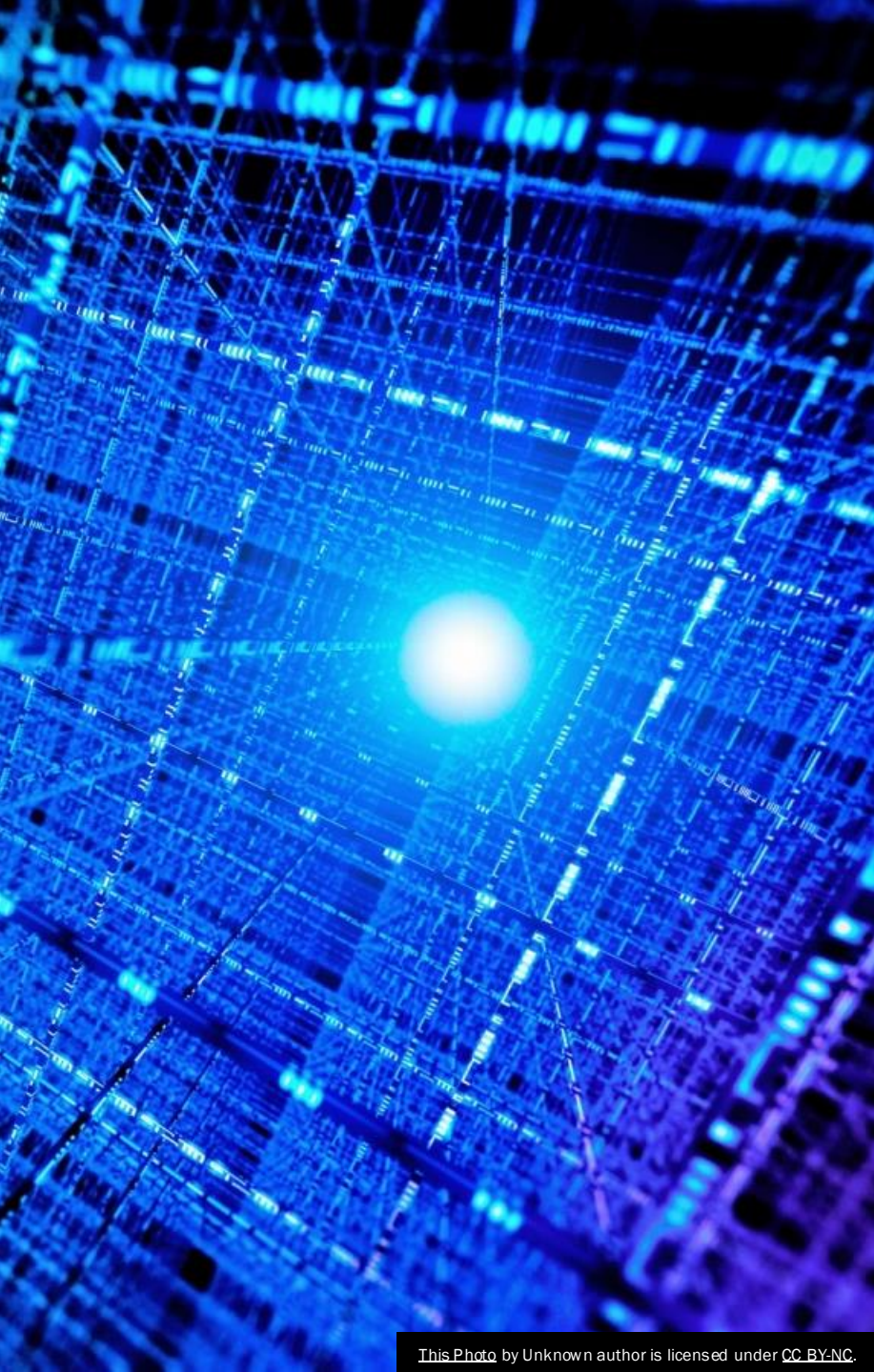


Model Compression – Pruning

- ❑ Technique originally used in decision trees
- ❑ Remove sections of a tree which are not important for classification
- ❑ In NNs pruning is done by setting the weights of some Neurons to zero.
- ❑ Reduces the number of non-zero parameters up to 90%
- ❑ Makes the neural networks more sparse
- ❑ Requires less storage space



[This Photo](#) by Unknown author is licensed under [CC BY-SA](#).

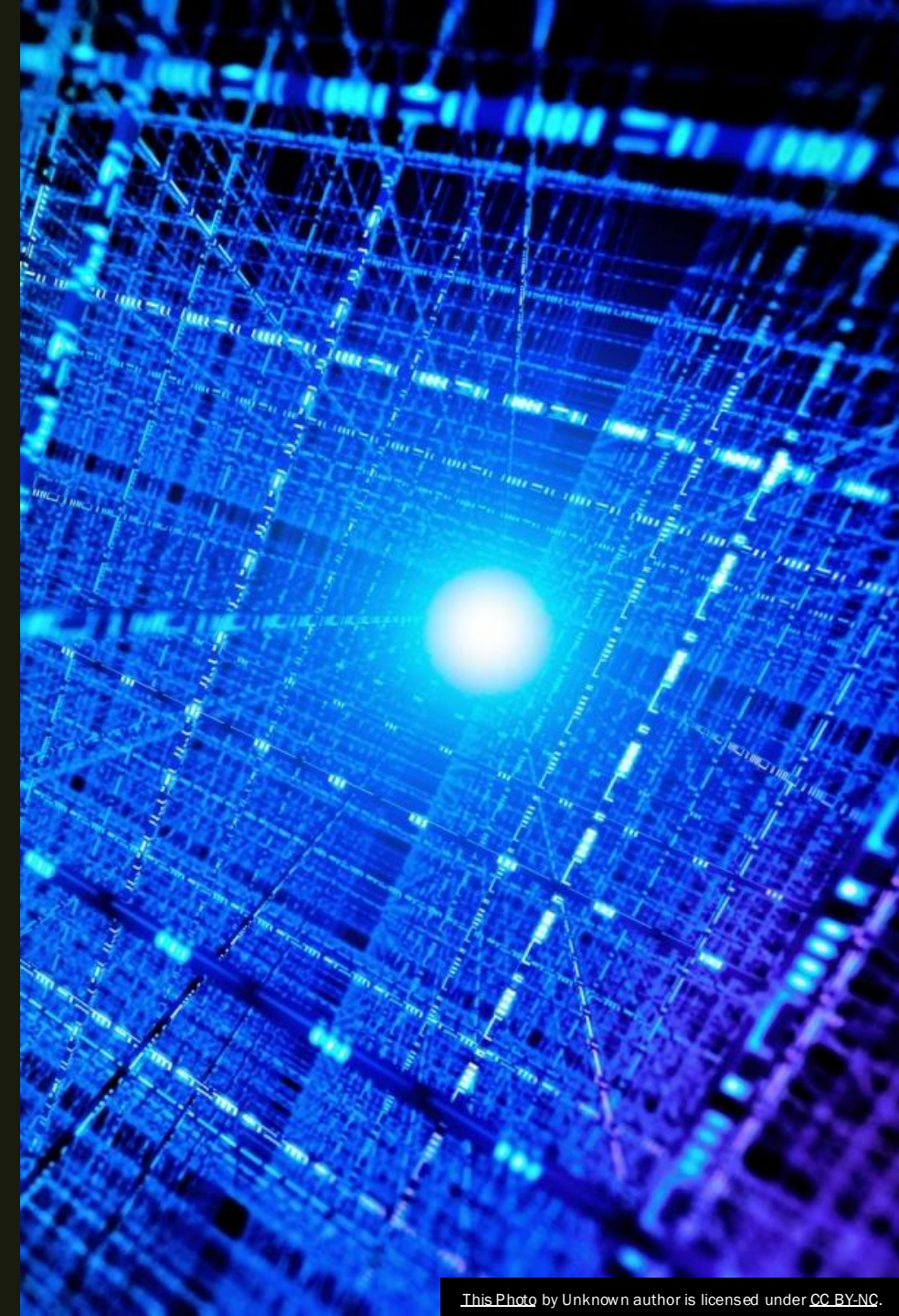
An abstract digital graphic on the left side of the slide. It features a complex, glowing blue and purple grid-like pattern that resembles a circuit board or a data visualization. A bright, circular white light source is positioned in the center-left, casting a soft glow over the surrounding pattern. The overall aesthetic is high-tech and futuristic.

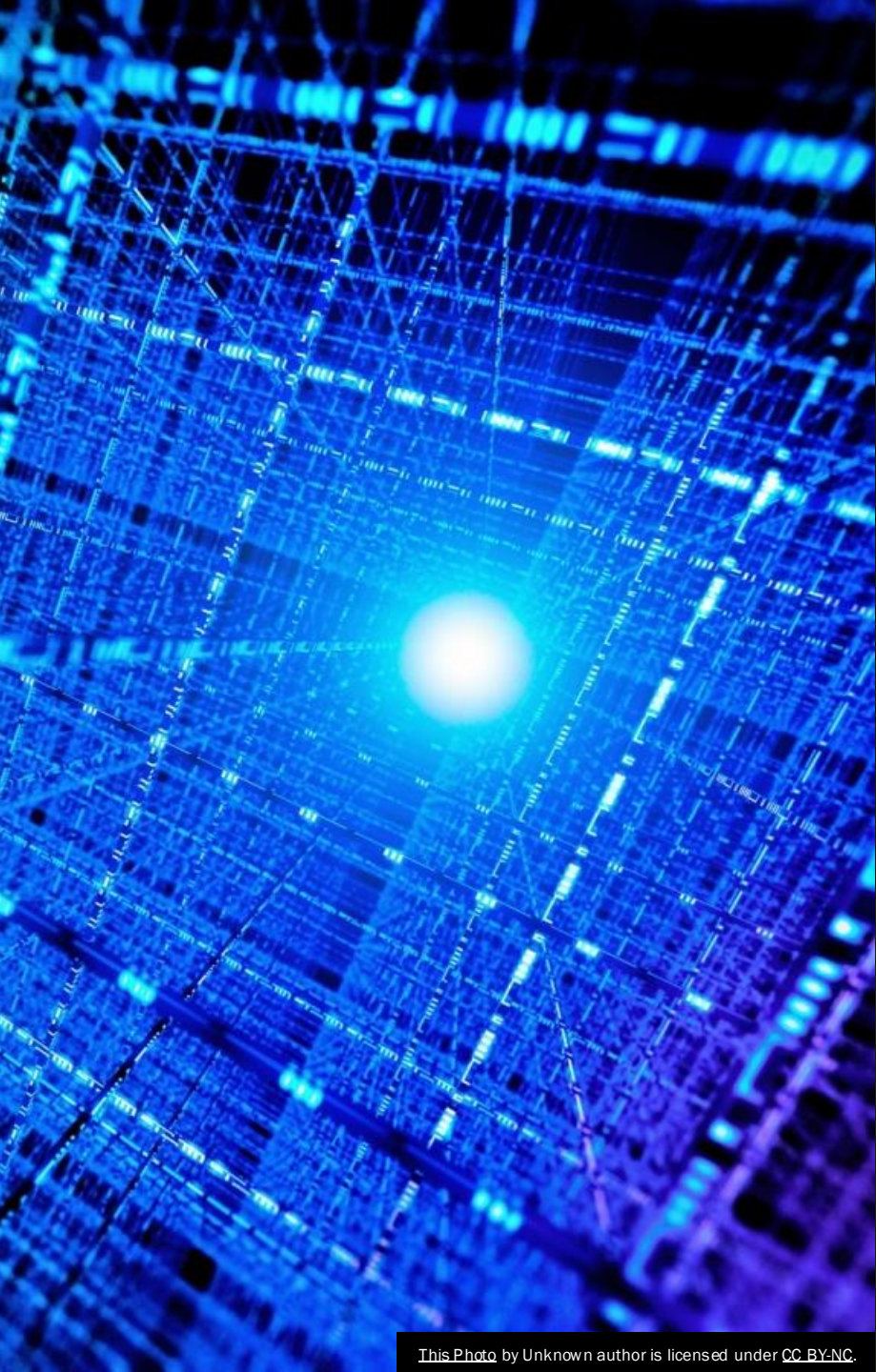
Model Compression – Quantization

- ❑ Most commonly used model compression method.
- ❑ Done by using fewer bits to represent the parameters
- ❑ Default is to use 32 bits for a float number (single precision)
- ❑ Model having 100M parameters would take up 400 MB
- ❑ Using 16 bits representation would reduce the size by half

Model Compression – Quantization

- ❑ It is also possible to use 8 bits integer representation (Fixed Point)
- ❑ Quantization also improves speed of computation
- ❑ Can lead to rounding errors and division by zero





Model Compression – Quantization

- ☐ Recent trend is to perform low precision training
- ☐ NVIDIA's Tensor Cores supports mixed precision training
- ☐ Google's TPUs supports 16-bit Brain Floating Point Format
- ☐ Fixed point (8 bits) training is still not available
- ☐ Fixed point inference is available on edge devices (Tensorflow Lite, PyTorch Mobile)

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

- A common dilemma in machine learning development: whether to prioritize accuracy or speed first when building a new model.
- Usually, one improve the accuracy first during the research phase.
- Roblox's journey to enhance inference speed of model BERT in production
- Objective - Text classification and Named Entity Recognition (NER) applications.
- Benchmark Metrics:
 - Latency – The medium time it takes to serve one request
 - Throughput – The number of requests served in one second
- For consistent comparison a single server with 36 Xeon Scalable Processor cores was used

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

GPU vs CPU Decision

- For model training GPU was much faster than CPUs
- For inference, GPUs scales the best in batch mode
- Cost economics of inference on CPU was better than GPU
- 3,000 inferences per second on an Intel Xeon Scalable 36-core server
- 400-500 inferences per second on a cost-equivalent Tesla V100 GPU

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

Thread Tuning

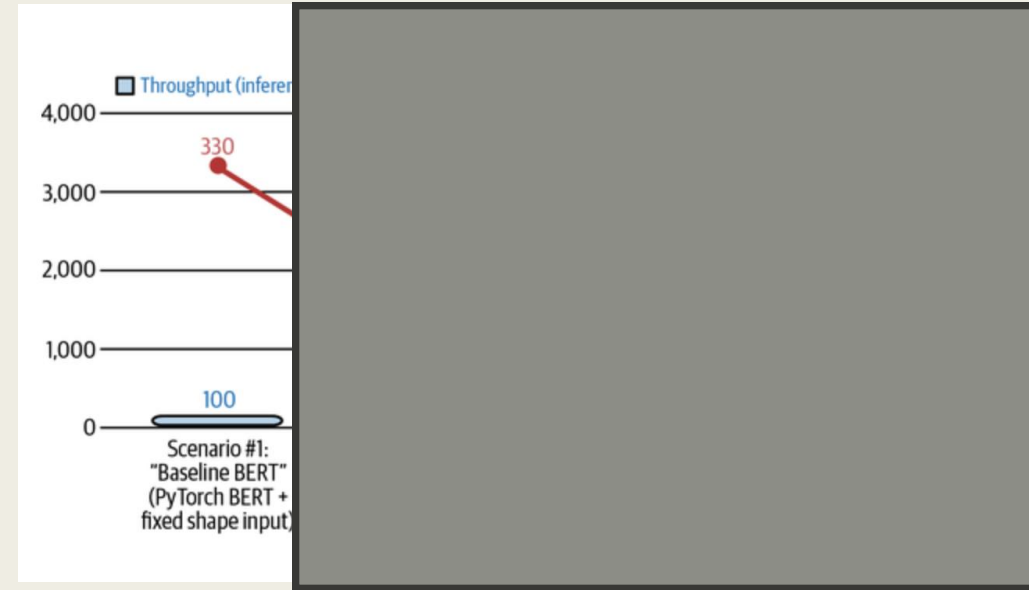
- For model training GPU was much faster than CPUs
- PyTorch must be properly thread-tuned before multiple worker processes can do concurrent model inference.
- Within each process, the PyTorch model attempted to use multiple cores to handle even a single inference request.
- This resulted in stagnation when too many of these workers were running at once in the same machine
- Set the number of threads to 1

```
1  import torch
2
3
4  class BertModelInference:
5      def __init__(self, model_path, do_quantize=False):
6
7          # Omitting code that loads the Bert model, for clarity
8
9          torch.set_num_threads(1)
```

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

- ❑ Started with BERT model with fixed shape input (128 tokens)

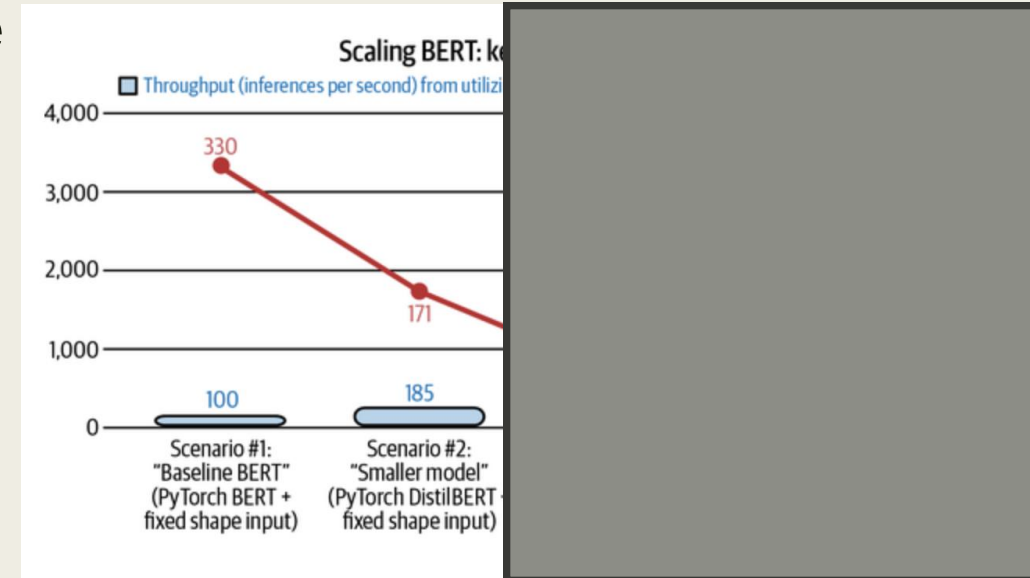


Source: Designing Machine Learning Systems, OREILLY

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

- ❑ Started with BERT model with fixed shape input (128 tokens)
- ❑ Replaced BERT with DistilBERT fixed shape input (128 tokens)

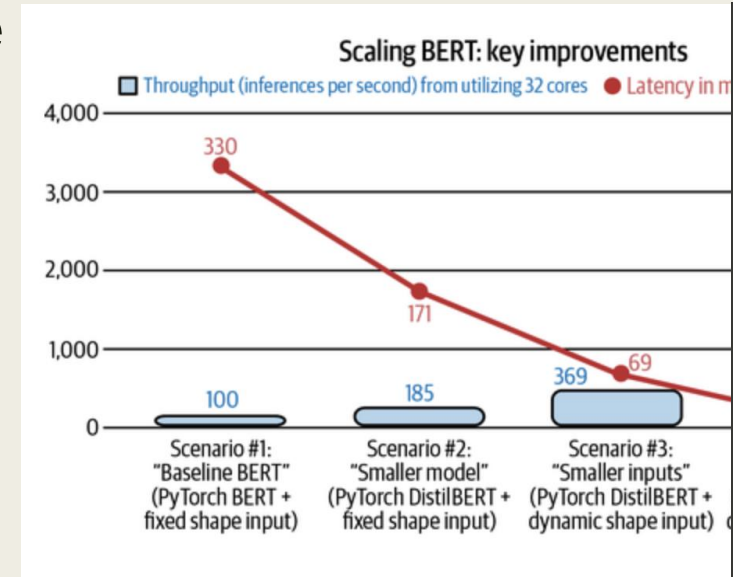


Source: Designing Machine Learning Systems, OREILLY

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

- ❑ Started with BERT model with fixed shape input (128 tokens)
- ❑ Replaced BERT with DistilBERT fixed shape input (128 tokens)
- ❑ Changed fixed shaped input to dynamic shape input (no padding with 0s)

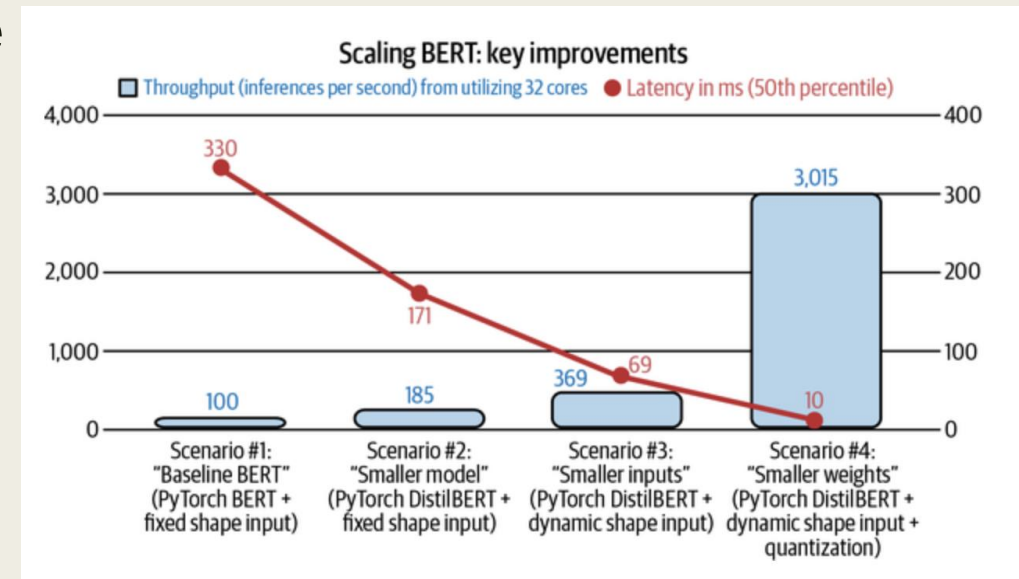


Source: Designing Machine Learning Systems, OREILLY

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

- ❑ Started with BERT model with fixed shape input (128 tokens)
- ❑ Replaced BERT with DistilBERT fixed shape input
- ❑ Changed fixed shaped input to dynamic shape input
- ❑ Finally implemented quantization



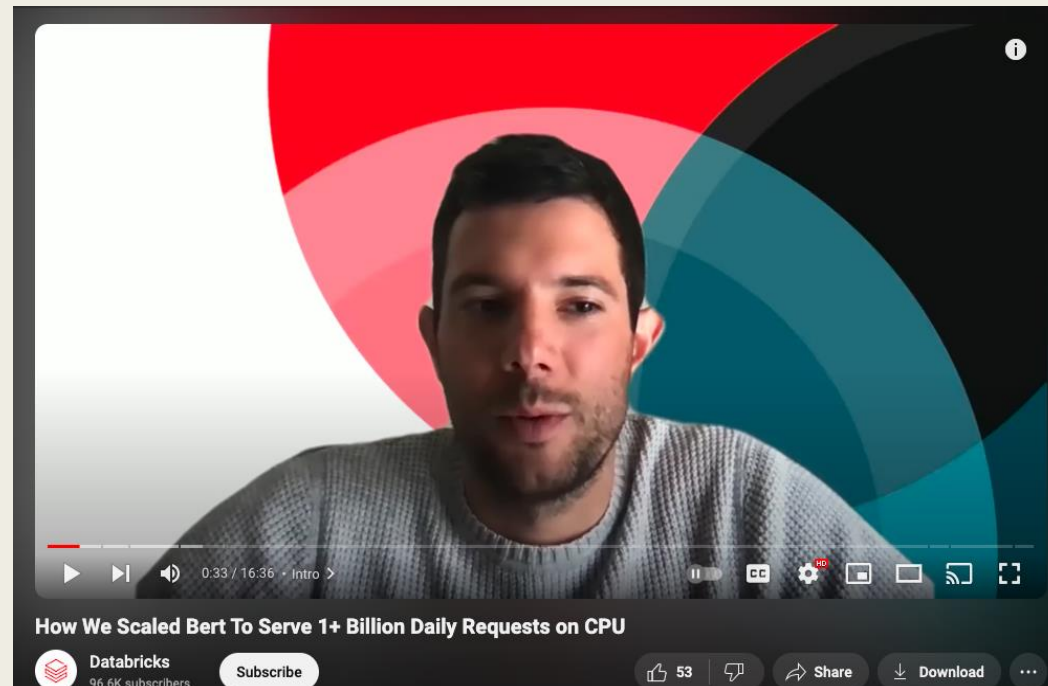
Source: Designing Machine Learning Systems, OREILLY

Model Compression - Case Study

How Roblox Scaled BERT to serve 1 Billion Daily Requests on CPUs

Home assignment – Watch the YouTube Video talk about this

https://www.youtube.com/watch?v=Nw77sEAn_Js



Source: YouTube

Summary of Today's Learning

- ❑ How to deploy ML models in production
- ❑ Comparision of Batch vs Online prediction scenarios
- ❑ Methods of compressing ML models